

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0614	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	N.SRIKANTH	Reg. No.:	192411139
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Q1. A disaster response application maps affected areas using coordinate points and determines the boundary using convex hull techniques. a) Explain how brute-force convex hull operates. b) Analyze its computational complexity. c) Discuss limitations when handling large datasets.

Q2. A cybersecurity system scans large log files to detect specific attack patterns using brute-force string matching. a) Explain how brute-force string matching works in this context. b) Analyze the time complexity of the algorithm. c) Discuss why performance issues arise with large logs.

Code of Conduct

I _____ N.SRIKANTH 192411139 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46
Overall Total (100)						86

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. 1. Disaster Response Application – Brute-Force Convex Hull

a) Explain how brute-force convex hull operates

The Brute-Force Convex Hull algorithm finds the outer boundary (convex hull) of a set of points by checking every pair of points.

Steps:

1. Select every pair of points.
2. Draw a line between them.
3. Check all remaining points.
4. If all other points lie on the same side of the line, then the line is part of the convex hull.
5. Repeat for all pairs until the boundary is formed.

Example:

Consider the points:

A(1,1), B(2,4), C(5,3), D(4,1), E(3,2)

- Check line AB → All other points are on one side → AB is a hull edge.
- Check line BC → Valid hull edge.
- Check line CD → Valid hull edge.
- Check line DA → Valid hull edge.
- Point E(3,2) lies inside the boundary, so it is not part of the convex hull.

Convex Hull = A → B → C → D → A

b) Computational Complexity

Assume there are n points.

- Number of pairs of points = $n(n-1)/2 = O(n^2)$
- For every pair, all remaining points are checked = $O(n)$

Therefore,

Overall Time Complexity

$$O(n^2) \times O(n) = O(n^3) \quad O(n^2) \times O(n) = O(n^3)$$

Space Complexity

$$O(1) \quad O(1) \quad O(1)$$

(excluding the space required to store the output.)

c) Limitations when handling large datasets

The brute-force convex hull algorithm is easy to understand and implement but is inefficient for large datasets.

Limitations include:

- It compares every pair of points, leading to excessive computations.
- Execution time increases rapidly as the number of points grows.
- It is unsuitable for real-time disaster management systems where quick decisions are essential.
- Processing thousands of GPS coordinates may take a long time.
- It wastes time checking many line segments that can never become hull edges.
- Modern algorithms such as Graham Scan, Jarvis March, and QuickHull provide much better performance for large datasets.

Q2.Cybersecurity System – Brute-Force String Matching

a) Explain how brute-force string matching works

In a cybersecurity system, log files generated by servers, firewalls, or intrusion detection systems contain millions of records. Security analysts often search these logs for specific attack signatures such as malicious commands, suspicious IP addresses, or malware patterns.

The Brute-Force String Matching Algorithm searches for a pattern by comparing it with every possible position in the text. It starts from the beginning of the log file and compares each character of the pattern with the corresponding characters in the log. If all characters match, the pattern is found. Otherwise, the pattern shifts by one position and the comparison starts again.

This process continues until the entire log file has been searched.

Working Procedure

1. Read the complete log file (text).
2. Read the attack signature (pattern).
3. Compare the pattern with the first characters of the text.
4. If characters match, continue comparing the remaining characters.
5. If a mismatch occurs, shift the pattern by one character.
6. Repeat the process until the end of the log file.
7. Report the location where the pattern is found.

b) Time Complexity

Let:

- n = Length of the text (log file)
- m = Length of the pattern

The algorithm compares the pattern at every possible position.

Best Case

When the first character mismatches immediately,

$O(n)$

Worst Case

When many characters match before a mismatch occurs,

$O((n-m+1) \times m)$

Approximately,

$O(nm)$

Space Complexity

$O(1)$

since only a few variables are used.

c) Why performance issues arise with large logs

Although brute-force string matching is simple and easy to implement, it performs poorly for very large log files.

Performance issues include:

- Every possible position in the text is examined.
- Many characters are compared repeatedly, causing redundant work.
- Large log files containing millions of records require enormous processing time.
- Searching for multiple attack signatures further increases execution time.
- The algorithm cannot meet the requirements of real-time intrusion detection systems where immediate threat detection is necessary.
- More efficient algorithms such as Knuth-Morris-Pratt (KMP), Boyer-Moore, and Rabin-Karp avoid unnecessary comparisons and provide much faster searching.

Summary Table

Algorithm	Purpose	Time Complexity	Example	Major Limitation
Brute-Force Convex Hull	Finds the outer boundary of coordinate points	$O(n^3)$	Boundary of flooded villages using GPS coordinates	Very slow for large numbers of points
Brute-Force String Matching	Searches for an attack pattern in a log file	$O(n \times m)$	Searching "FAILED" in server logs	Repeated comparisons make it slow for large log files